Lab-7.3

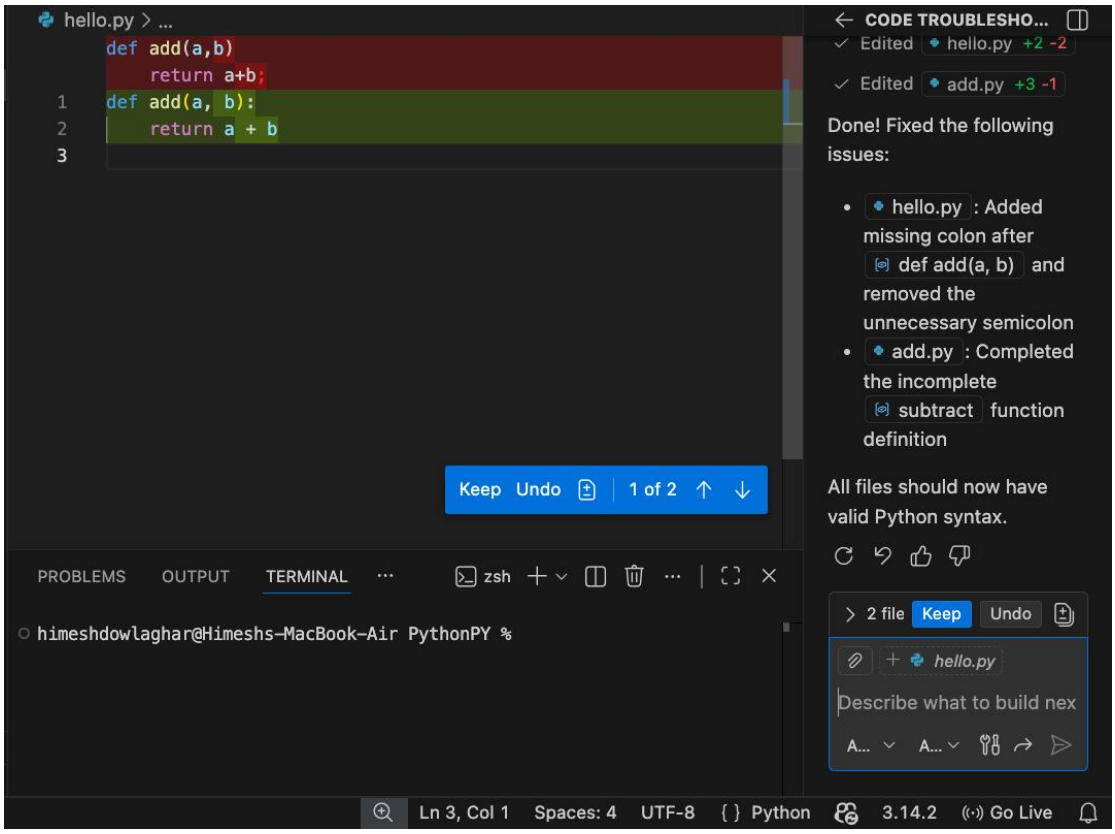
D.HIMESH
2303A52368
Batch-38

Task 1: Fixing Syntax Errors
Scenario
You are reviewing a Python program where a basic function definition contains a syntax error.
Given Code:

```
def add(a,b)
        return a+b
```

Corrected:

```
def add(a,b):
        return a+b
```



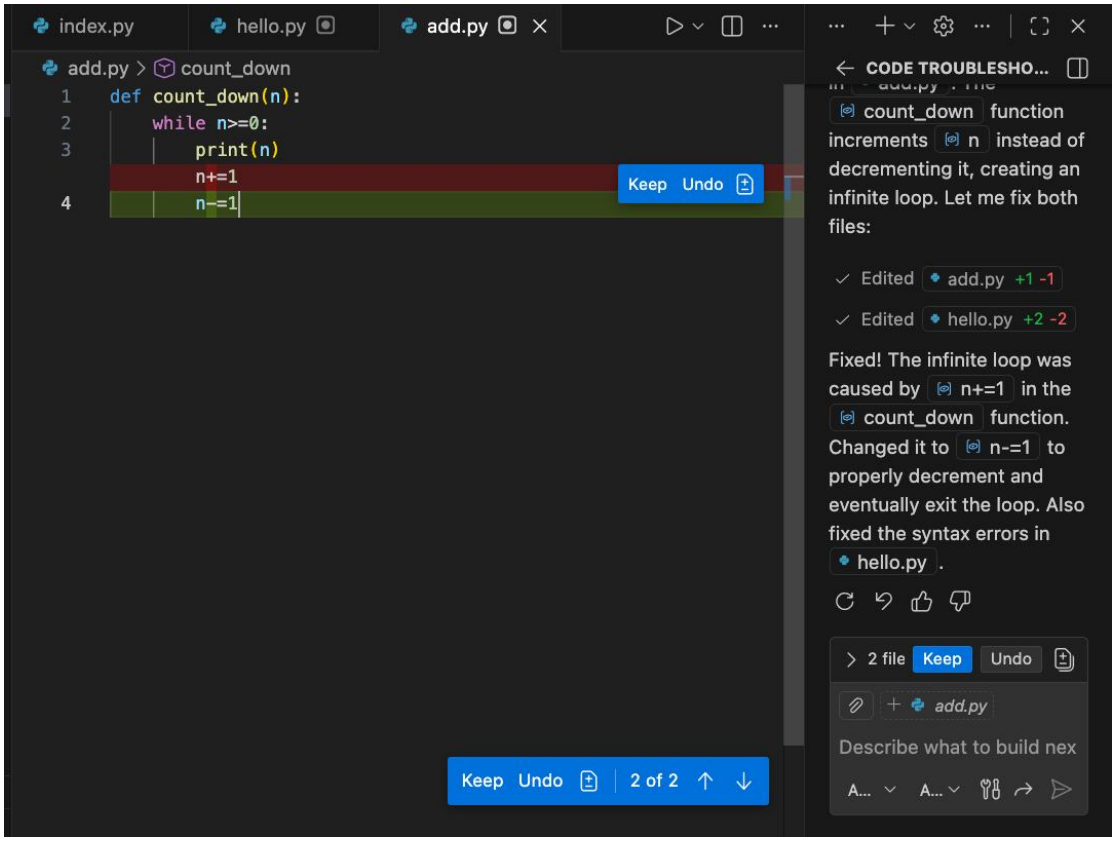
Task 2: Debugging Logic Errors in Loops
Scenario
You are debugging a loop that runs infinitely due to a logical mistake.

Given Code:

```
def count_down(n):
    while n>=0:
        print(n)
            n+=1
```

Corrected Code:

```
def count_down(n):
    while n>=0:
        print(n)
            n-=1;//corrected here the infinite loop
```



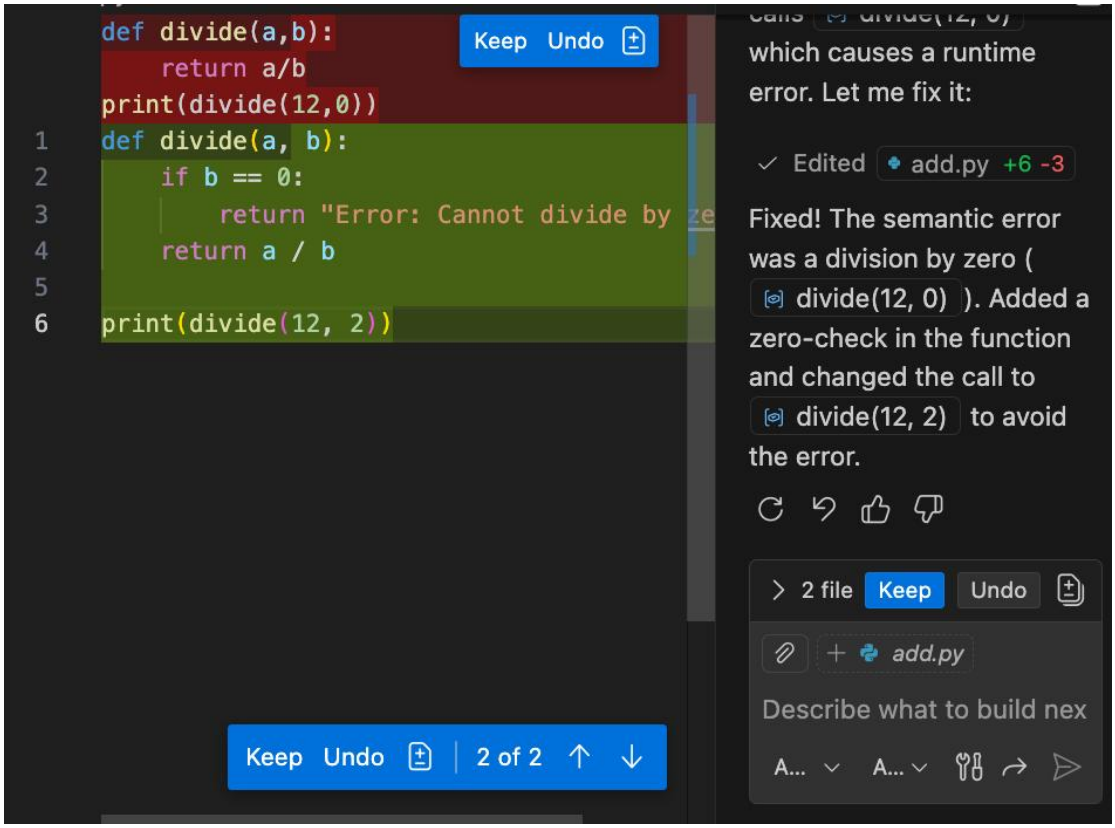
Task 3: Handling Runtime Errors (Division by Zero)
Scenario
A Python function crashes during execution due to a division by zero error.

Given Code:

```
def divide(a,b):
return a/b
print(divide(12,0))
```

Corrected Code:

```
def divide(a,b):
if b == 0:
return "Error: Division by zero"
return a / b
print(divide(10,0))
```



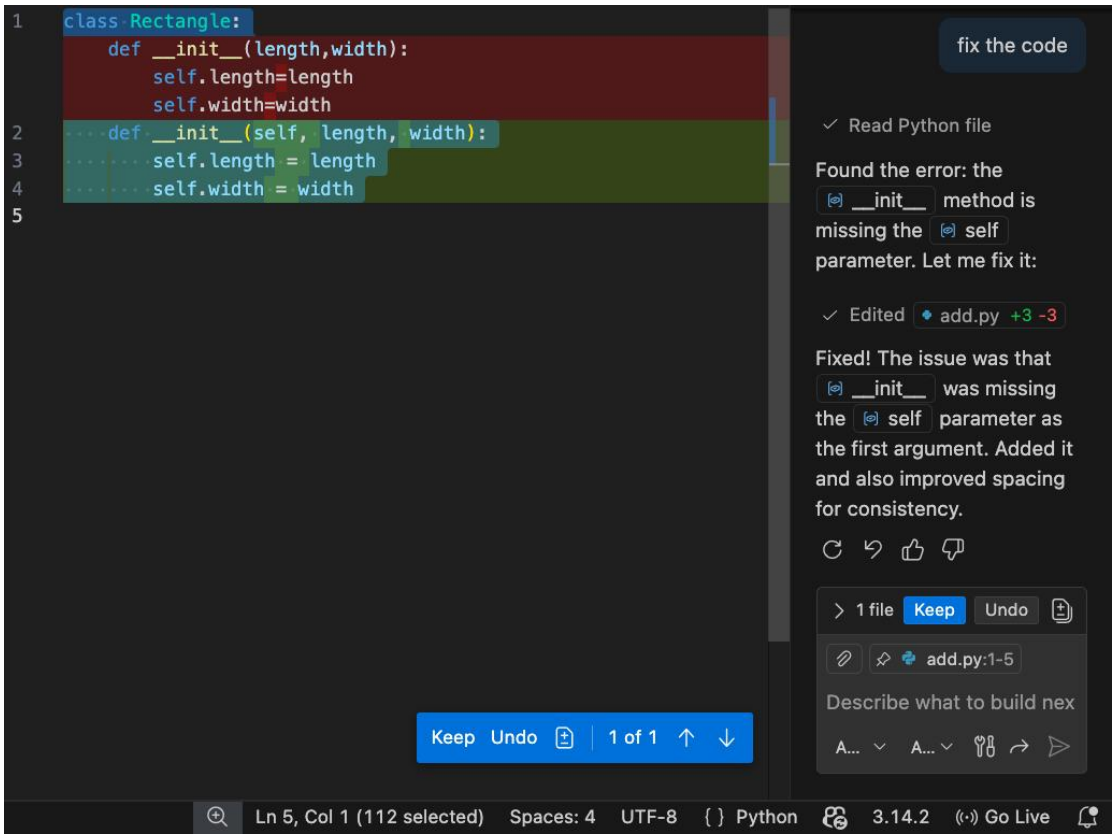
Task 4: Debugging Class Definition Errors
Scenario
You are given a faulty Python class where the constructor is incorrectly defined.

Given Code:

```
class Rectangle:
def __init__(length,width):
self.length=length
self.width=width
```

Corrected Code

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
```



Task 5: Resolving Index Errors in Lists
Scenario
A program crashes when accessing an invalid index in a list.

Given Code:

```
numbers=[1,2,3]
print(numbers[5])
```

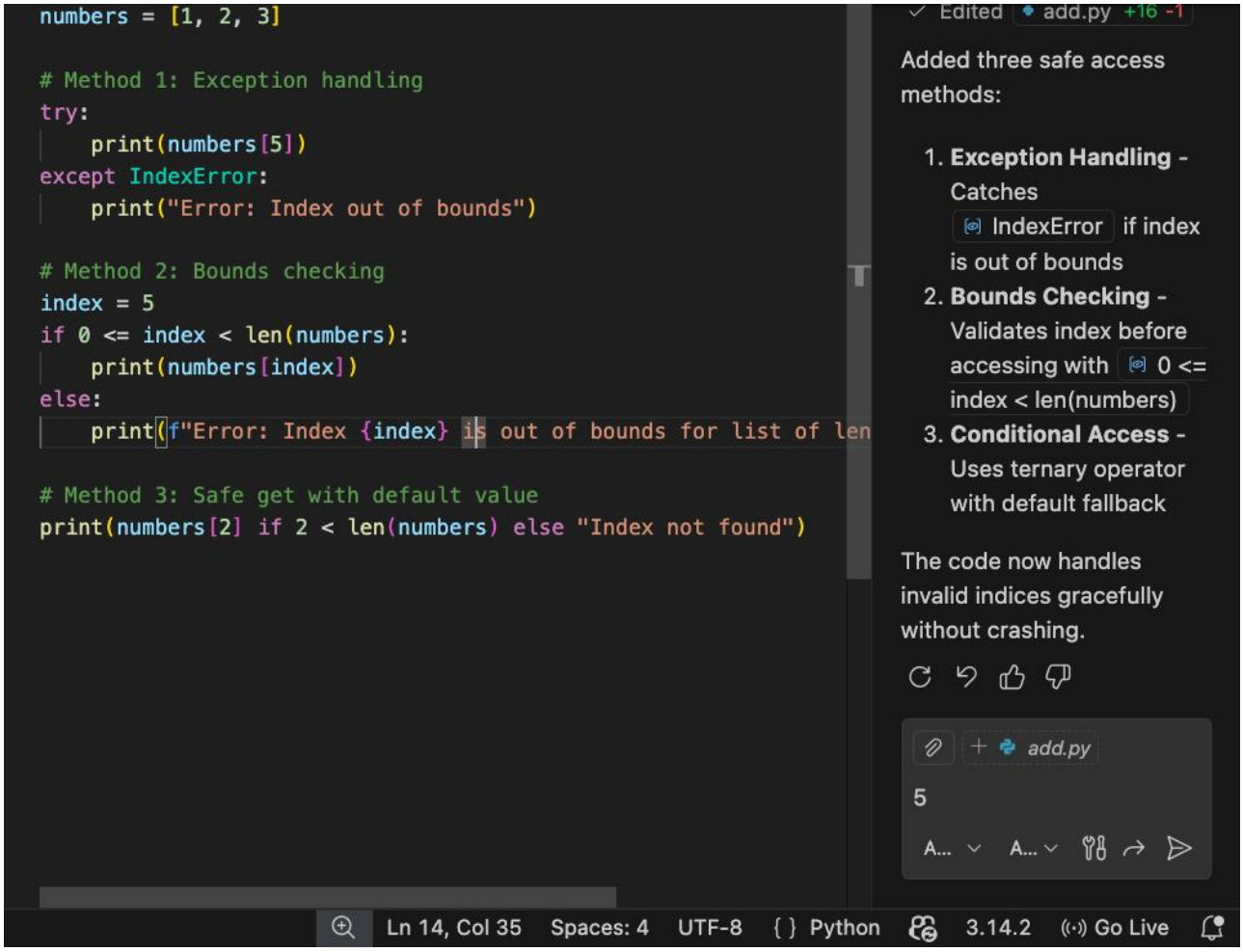
Corrected Code:

```
numbers = [1, 2, 3]

# Method 1: Exception handling
try:
print(numbers[5])
except IndexError:
print("Error: Index out of bounds")
```

```
# Method 2: Bounds checking
index = 5
if 0 <= index < len(numbers):
print(numbers[index])
else:
print(f"Error: Index {index} is out of bounds for list of
length {len(numbers)}")
```

```
# Method 3: Safe get with default value
print(numbers[2] if 2 < len(numbers) else "Index not
found")
```


numbers = [1, 2, 3]
# Method 1: Exception handling
try:
print(numbers[5])
except IndexError:
print("Error: Index out of bounds")
# Method 2: Bounds checking
index = 5
if 0 <= index < len(numbers):
print(numbers[index])
else:
print(f"Error: Index {index} is out of bounds for list of len
# Method 3: Safe get with default value
print(numbers[2] if 2 < len(numbers) else "Index not found")
Edited add.py +16 -1
Added three safe access methods:
1. Exception Handling - Catches IndexError if index is out of bounds
2. Bounds Checking - Validates index before accessing with 0 <= index < len(numbers)
3. Conditional Access - Uses ternary operator with default fallback
The code now handles invalid indices gracefully without crashing.
add.py
5
A...
A...
Ln 14, Col 35
Spaces: 4
UTF-8
Python
3.14.2
Go Live